Exposio — what is implemented, and how it works

Implementation brief for the supervisor meeting

Author	Esmail Mohammed Salem Abdullah — MSc Cyber Security Engineering, Óbuda University
Supervisor	Balázsné Dr. Kail Eszter
Date	11 October 2026
Live system	https://exposurescope.vercel.app (own domain: exposio.eu)
Source	https://github.com/SocoSec/exposurescope

1. Summary

Since the draft you reviewed, the system has been implemented and deployed. It is not a prototype of selected parts: the complete path runs in production — a user verifies ownership of an address by one-time code, the address is looked up against breach intelligence, the Breach Risk score is computed server-side, a ranked remediation plan is produced, and an advisory agent answers questions about the result. A live lookup against a real address currently returns 271 breach records.

Two things have been added that were not in the draft at all. First, an adversarial test suite that assumes the language model is attacker-controlled and measures what the application still prevents. Second, a sensitivity analysis of the scoring model, which addresses your objection that the parameter values are unjustified. Both are reproducible offline, without keys or network access (§7).

This brief also records three numerical errors I found in the draft while verifying it against the implementation (§8). I would rather raise them than have them found.

Status at a glance

Measure	Value
API routes	4 — verify/start, verify/check, breach, chat
Library modules	13 — 7 domain, 6 platform
Runtime dependencies	next, react, react-dom. No ORM, no database driver, no ML runtime
Correctness tests	41 passing
Adversarial tests	18 attacks, 0 escaped
Persistent storage	None. Rate-limit counters only, keyed by HMAC hash, expiring in 24 h
Live breach lookup	271 records returned for a verified address

2. How the application works

The flow has three phases. **Phase A — ownership.** The user submits an address and a six-digit code is emailed to it. The server does not retain the code: it returns a signed *challenge* token containing a keyed tag derived from the code, valid ten minutes. When the user enters the code, the server recomputes the tag, compares it in constant time, and issues a *scan* token valid fifteen minutes and bound to a keyed hash of that specific address.

Phase B — lookup and scoring. The scan token is presented with the address. The server recomputes the keyed hash and refuses any token that does not match, so a token issued for one address cannot be used to query another. Breach providers are then queried in parallel, results are deduplicated across providers,

and the score is computed from the merged set. The result is returned together with a signed *report* token valid one hour. Nothing is written to disk.

Phase C — advice. The user may ask the advisory agent questions. The agent receives no free text from the browser describing the report; it works from the signed report token and calls tools that execute project code. Its output is validated before display.

Password checking is a separate path that does not involve the server at all: the password is hashed in the browser and only the first five characters of the hash are sent to the Pwned Passwords range API, with the comparison completed locally.

3. Request lifecycle

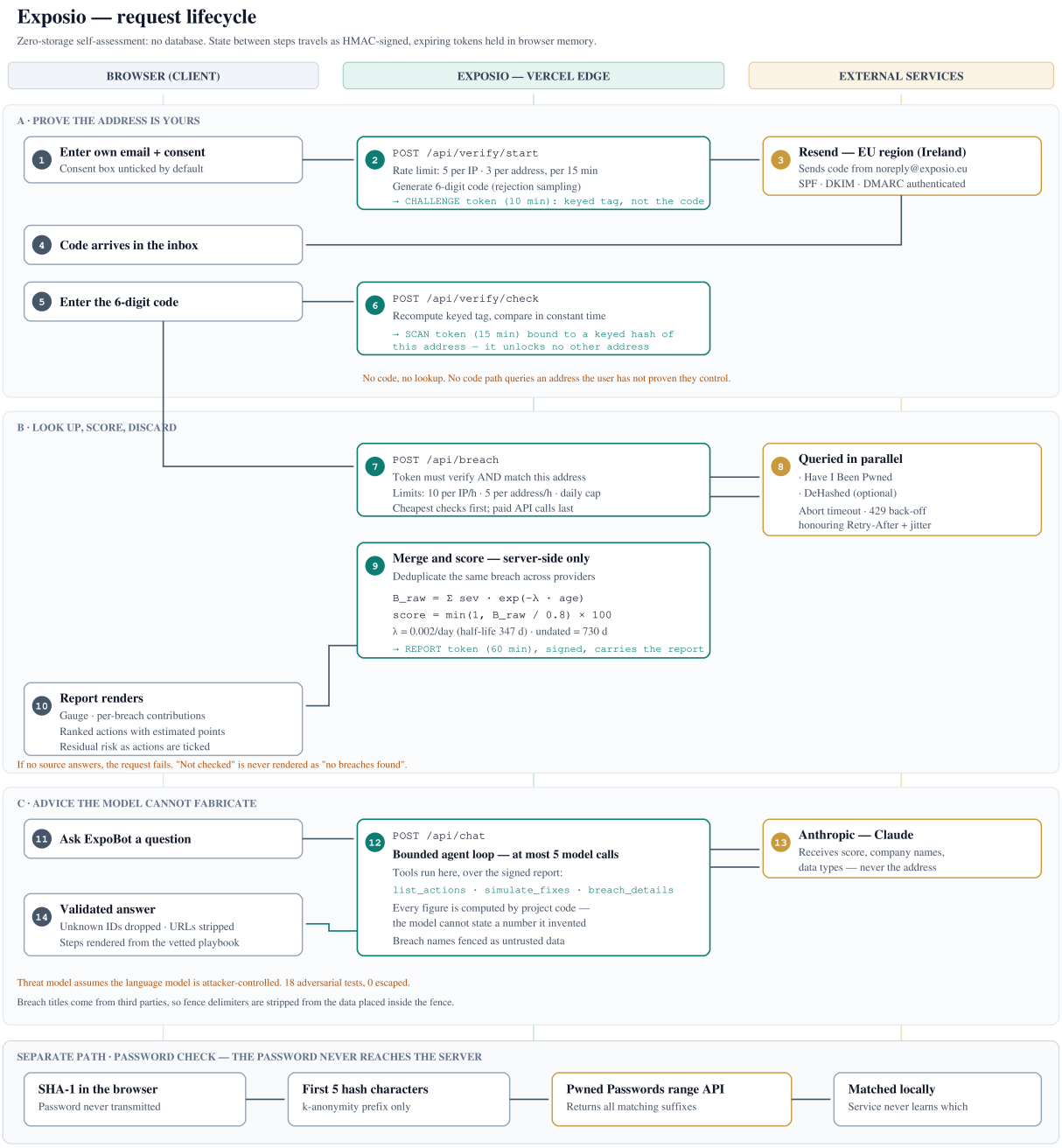


Figure 1. Request lifecycle. Ownership verification precedes any lookup; the password path never reaches the server.

4. Architecture

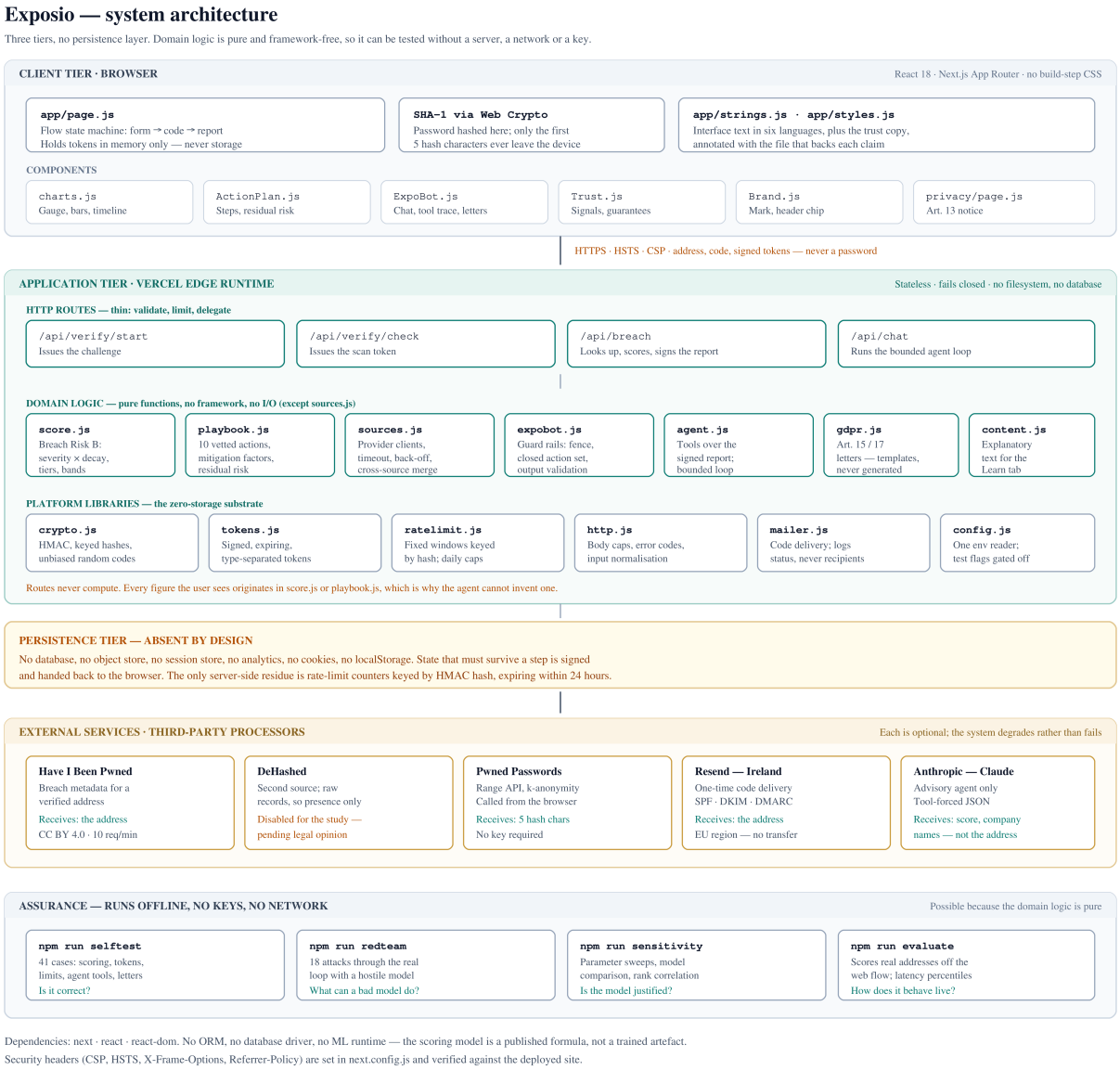


Figure 2. System architecture. The persistence tier is drawn in the position a database would occupy, to show that its absence is structural.

The separation that matters is between the routes and the libraries. Routes validate input, apply limits and delegate; they contain no computation. Every figure a user sees originates in `score.js` or `playbook.js`, which are pure functions with no framework dependency and no I/O. That is what allows the whole domain model to be tested offline, and it is also why the advisory agent cannot invent a number: it has no arithmetic of its own to offer.

5. Implementation decisions

This section responds to the comment that the implementation description is superficial. Each item below is a decision with a reason and a consequence, rather than a description of a feature.

5.1 Where state lives, given that nothing is stored

Zero storage is a requirement, not an outcome, so state between steps had to go somewhere. It travels as HMAC-SHA256 signed tokens held in browser memory, in three types with different lifetimes: challenge (10 min), scan (15 min), report (60 min). The signature is verified *before* the payload is parsed, so a malformed token is rejected without the parser ever seeing attacker-controlled structure. The type is part of the signed payload, so a scan token presented where a report token is expected fails verification — this is asserted by a test, because it is exactly the mistake that would let a user bypass the lookup and reach the agent directly.

5.2 Binding a lookup to a verified address

A token proving *some* verification occurred is insufficient; it must prove verification of *this* address. The scan token therefore carries a keyed hash of the address rather than the address itself. On lookup the server recomputes that hash from the submitted address and compares. The consequence is that there is no code path, privileged role or configuration flag that performs an unverified lookup — the property is structural rather than a policy, which is the substantive difference between this system and an OSINT tool.

5.3 Checking a password without receiving it

The password is hashed with SHA-1 in the browser using the Web Crypto API. Only the first five hexadecimal characters of the hash are transmitted; the service returns every stored suffix sharing that prefix, and the match is completed locally. The service therefore learns that someone checked a password beginning with a given prefix — a set large enough to be uninformative — and never learns which. This is the k-anonymity construction used by Pwned Passwords, and it is the reason the password feature can exist at all in a tool whose central claim is that it stores nothing.

5.4 Two providers with different latency and rate limits

Providers are queried concurrently, so the user waits for the slower one rather than the sum. Each call is bounded by an AbortController timeout so a single slow provider cannot hold the request open. On HTTP 429 the client honours the Retry-After header, adds jitter so simultaneous retries do not re-collide, and retries once — but abandons the source if the required wait exceeds three seconds, on the basis that a correct answer delivered too late is not a correct answer. A failed provider does not fail the request: the report is produced from whichever sources answered, and the interface names those that did not.

One rule is enforced above all: if *no* source answers, the request fails. “Not checked” is never rendered as “no breaches found”. This is currently demonstrated rather than argued, because DeHashed has been returning HTTP 403 throughout — the account's search subscription is inactive — while reports continue to render correctly from HIBP alone.

5.5 An advisory agent that cannot fabricate security advice

This received the most engineering effort, because a language model that invents a settings path or a recovery phone number in a security tool causes real harm. The design does not ask the model to be correct; it removes its ability to express an incorrect answer.

Control	Mechanism
Closed action set	Actions referenced only by identifier, from an enumeration generated per report
Computed figures	simulate_fixes calls residualScore(); the model cannot state a number it invented
Human-authored text	Remediation steps come from playbook.js, written by hand and versioned in git

Untrusted data fenced	Third-party breach names delimited; delimiters stripped from the data itself
Output validated	Unknown identifiers dropped, URLs removed, length and counts capped
Bounded execution	At most five model calls per question; a cap on tool calls per round

The evaluation method matters as much as the controls. A hostile model implementation is injected into the production agent loop — the same function the live route executes — and eighteen attacks are asserted to be contained: invented actions, phishing links, GDPR letters addressed to companies the user was never breached in, budget exhaustion, tool abuse, and indirect prompt injection. Because the threat model assumes the model is attacker-controlled, the result is a property of the application rather than of whichever model version is deployed.

The suite found a genuine defect, which I report rather than having quietly fixed. Two injection cases failed when first written: a breach title containing the fence delimiter closed the untrusted-data block, so the remainder of the title was read as instruction rather than data. Since DeHashed returns uncurated database names taken from leaked dumps, the input path was attacker-reachable. The two cases are now regression tests.

6. The scoring model and its parameters

You observed that the formula, weights and decay constant are presented without justification, empirical validation, sensitivity analysis or comparison with alternatives. I have built the analysis rather than added argument. It runs as `npm run sensitivity` over 2,000 synthetic identities with a fixed seed, and it begins by scoring every identity twice — once with its own parameterised implementation and once with the deployed `score.js` — asserting zero mismatches, so the analysis provably describes the real model.

6.1 What the constants define

The defensible justification is not that the values were fitted, but that each one defines something that can be stated and defended. λ = 0.002/day gives a half-life of 347 days: the model's claim is that a breach's contribution to present risk halves roughly every year. B_{max} = 0.8 fixes the top of the scale, and the analysis makes that concrete — a score of 100 corresponds to one breach exposing payment data together with passwords, approximately 112 days old. The band boundary at 66 corresponds to one credential breach about 141 days old.

6.2 Sensitivity

Parameter varied	Effect on ranking (Spearman)	Interpretation
λ across a 16-fold range	0.606 – 1.000, ≥ 0.90 over the plausible range	Governs the level of the score, not the order
B■■■ from 0.4 to 1.5	≈ 1.000 throughout	A monotone rescaling; a presentation decision
Undated-record age	falls to 0.769 at the extreme	The most consequential free parameter
Tier weights perturbed ±30%	mean 0.996, 95% keep their band	Ordering is robust to substantial weight error

The last row is the direct answer to the objection. The claim is not that 1.0 / 0.7 / 0.4 / 0.2 are the correct weights — they are informed estimates grounded in the account-takeover literature. The claim is that the conclusions a user draws survive those weights being substantially wrong.

6.3 Comparison with alternative schemes, and a finding against my own model

Alternative weighting scheme	Spearman ρ	Kendall τ-b	Band agreement
Count of breaches only	0.641	0.470	63.3%
Severity, no recency decay	0.659	0.478	50.8%
Single worst breach only	0.966	0.880	83.8%
Linear decay over 10 years	0.926	0.774	63.4%
Binary credential flag	0.405	0.321	43.4%

Severity weighting and recency decay are each doing real work: simply counting breaches produces a materially different ordering (ρ = 0.641), as does flagging credentials (ρ = 0.405). That answers the obvious question of why not simply count.

The third row is less comfortable and I think it is the most important result. Scoring by each person's single worst breach reproduces the full model almost exactly. The concentration analysis explains why: the largest single contributor is a median 92% of the raw score, and for 59% of identities one breach accounts for more than 80% of it. Exponential decay suppresses older records so aggressively that the summation adds little over a maximum. I would like to discuss whether to justify the aggregation by quantifying the cases where the two diverge, adopt the simpler model, or reduce λ so that aggregation does more work.

7. What is not implemented

Stated explicitly, because the draft describes components that do not exist and I would rather narrow the claims than have the gap discovered. A search of the repository for these terms returns nothing, and the only runtime dependencies are next, react and react-dom.

Component	Status
GBDT risk model	Not implemented — scoring is a deterministic published formula
BERT-BiLSTM NER pipeline	Not implemented — no text-analysis component exists
GNN identity alignment	Not implemented — no graph component exists
Shodan integration	Not implemented
Exposure Score components O, S, M, L	Not implemented — the interface says 'Breach Risk score', not 'Exposure Score'
DSAR automation	Letters are generated for the user to send; the system contacts no company

On the first three: the deterministic formula was chosen over a learned model deliberately, and I would argue it is the better choice for this problem. A published formula can be recomputed by the user, audited by a reviewer and reproduced by a test. A gradient-boosted model would remove all three properties, and there is no labelled outcome data to train it against.

8. Corrections to the draft

Location	Issue
§5.3, worked example 1	Two recency terms are roughly 100× too large. The raw sum is 0.5123, not 0.563, so the score is 64, not 70. The c
§5.4, worked example 2	Uses a default recency of 0.05; the implementation assumes 730 days, giving 0.2322. The score is 46 (Medium), n
§5.2	The undated-record assumption is not stated. §6.2 shows it is the model's most consequential free parameter.
Table 3	Three bands are defined; the implementation has four — a distinct 'safe' band at zero.

9. How to verify any of this

All three run offline, with no API keys and no network access:

```
git clone https://github.com/SocoSec/exposurescope
cd exposurescope && npm install
npm run selftest # 41 passed, 0 failed
npm run redteam # 18 attacks contained, 0 escaped
npm run sensitivity # the tables in section 6
```

The live system is at https://exposurescope.vercel.app. Supporting documents in the repository: docs/IMPLEMENTATION-STATUS.md (component-by-component), docs/SCORE-MODEL-ANALYSIS.md (the full sensitivity findings), docs/TEST-SPECIFICATION.md (all 59 test cases, mapped to GDPR articles and the OWASP Top 10 for LLM Applications).

What I would like to discuss. First, whether to keep the summation in the scoring model or adopt the simpler maximum, given §6.3. Second, how far to narrow the written claims to match §7. Third, whether an expert-ranking exercise — five to eight practitioners ranking fifteen anonymised profiles, compared with the model by rank correlation — is worth running before submission; it is the cheapest route from a defensible model to a validated one.